Simplified Examiner Architecture

React Frontend
↓
Gemini API
↓
Interview Engine

Whisper
↓
Speech To Text

Coqui TTS
↓
Avatar Voice

MediaPipe
↓
Eye Contact + Posture

Librosa
↓
Confidence Analysis

DeepFace
↓
Emotion Analysis

Final Report Generator

No backend database required.

What I Would Build Now

Phase 1 (Highest Priority)

Gemini Interview Engine

When user clicks:

Enter Interview

Send prompt to Gemini:

You are a professional interview panel.

There are three interviewers:

1. Technical Interviewer
2. HR Interviewer
3. Domain Expert

Rules:

- Ask one question at a time.
- Wait for candidate response.
- Evaluate response.
- Generate follow-up questions.
- Rotate interviewers:
 Technical → HR → Domain Expert
- Conduct 10-15 questions total.
- At the end generate a final evaluation.

Store conversation in React state only:

const [messages, setMessages] = useState([]);

No database.

Phase 2

Voice Input

Browser already supports:

MediaRecorder

Flow:

User Speaks
↓
Audio Blob
↓
Whisper
↓
Text
↓
Gemini

You can even use Whisper API directly.

No backend needed.

Phase 3

Make Interviewers Speak

Current:

Avatar
↓
Text Question

Target:

Avatar
↓
Speaks Question

Flow:

Gemini Question
↓
Coqui TTS
↓
Audio
↓
Play Audio
↓
Lip Sync Avatar

This will massively increase immersion.

Phase 4

Real Multi-Interviewer System

Right now you have:

Avatar 1
Avatar 2
Avatar 3

but they don't have identities.

Create personalities:

```
const interviewers = {
 technical: {
   name: "Alex",
   role: "Technical Interviewer"
 },

 hr: {
   name: "Sophia",
   role: "HR Interviewer"
 },

 expert: {
   name: "David",
   role: "Domain Expert"
 }
};
```

Question flow:

Alex:
Explain JVM.

Sophia:
Tell me about a challenge in your project.

David:
How would you scale this system?

This is the feature that makes Examiner different from normal interview bots.

Phase 5

Final Report

At interview end:

Send entire conversation to Gemini.

Prompt:

Analyze this interview.

Generate:

Technical Score (0-100)

Communication Score (0-100)

Confidence Score (0-100)

Strengths

Weaknesses

Improvement Suggestions

Overall Rating

Gemini returns JSON:

```
{
 "technicalScore": 82,
 "communicationScore": 76,
 "confidenceScore": 70,
 "strengths": [
   "Good Java fundamentals"
 ],
 "weaknesses": [
   "Needs system design knowledge"
 ]
}
```

Display in your analytics page.

No database required.

Phase 6 (After MVP Works)

Add AI Analysis:

MediaPipe

Eye Contact
Head Movement
Posture
Attention

Librosa

Speaking Speed
Pause Frequency
Voice Stability
Confidence

DeepFace

Stress
Confusion
Confidence
Emotion

Merge these scores with Gemini's evaluation.

What I Would Remove Completely

❌ Spring Boot

❌ MySQL

❌ Login/Register

❌ User Profiles

❌ Interview History

❌ Session Storage Database

❌ Admin Panel

For a college project, these add complexity but don't improve the core experience.

The next thing you should build this week

1. Connect Gemini API.
2. Start interview button generates first question.
3. User answers.
4. Gemini generates next question.
5. Rotate between the 3 interviewers.

6. After 10–15 questions generate final report.

Once those 6 steps work, Examiner becomes a complete AI interview platform rather than a frontend demo.